RAPPORT DE TRAVAUX PRATIQUES

FACTORY METHOD

Étude et implémentation en Java

Étudiant : Togyanouba Moïse

Matricule : XXXXXXX

Unité d'enseignement : SIGL

Lien Github : https://github.com/organizations/INF-461/settings/profile

Code :INF 461

Établissement : Université de Yaoundé 1

Année académique : 2026- 2027

Niveau : Master 1

Projet réalisé en langage Java

1. Introduction

Ce travail pratique porte sur la création d'objets à partir d'une fabrique en Java. L'objectif est de regrouper la logique de création des produits dans une classe dédiée. Le client demande un produit sans devoir construire directement les classes concrètes.

Le projet comporte deux versions. La première est celle du cours, qui prend en charge les produits A, B et C. La deuxième est une version modifiée dans laquelle le produit D est ajouté et certains éléments du code sont améliorés.

2. Présentation des classes

Classe	Rôle
Client	Demande la création des produits et utilise leur méthode commune.
FabriqueProduit	Centralise la création des objets selon le type reçu.
Produit	Classe abstraite qui définit le contrat commun des produits.
ProduitA	Implémentation concrète du produit A.
ProduitB	Implémentation concrète du produit B.
ProduitC	Implémentation concrète du produit C.
ProduitD	Nouveau produit ajouté dans la version modifiée.

La classe Produit

Produit est une classe abstraite. Elle déclare la méthode methodeproduit(), que chaque classe concrète doit implémenter. Cela permet au client de manipuler les différents produits à travers un même type.

La classe FabriqueProduit

La fabrique possède une méthode createProduit(String type). Elle reçoit une chaîne de caractères et utilise une instruction switch pour déterminer quelle classe concrète instancier.

La classe Client

Le client appelle createProduit(), récupère un objet de type Produit, puis appelle methodeproduit(). Il n'a donc pas besoin de connaître les détails internes de chaque classe concrète.

3. Diagramme UML de la version du cours

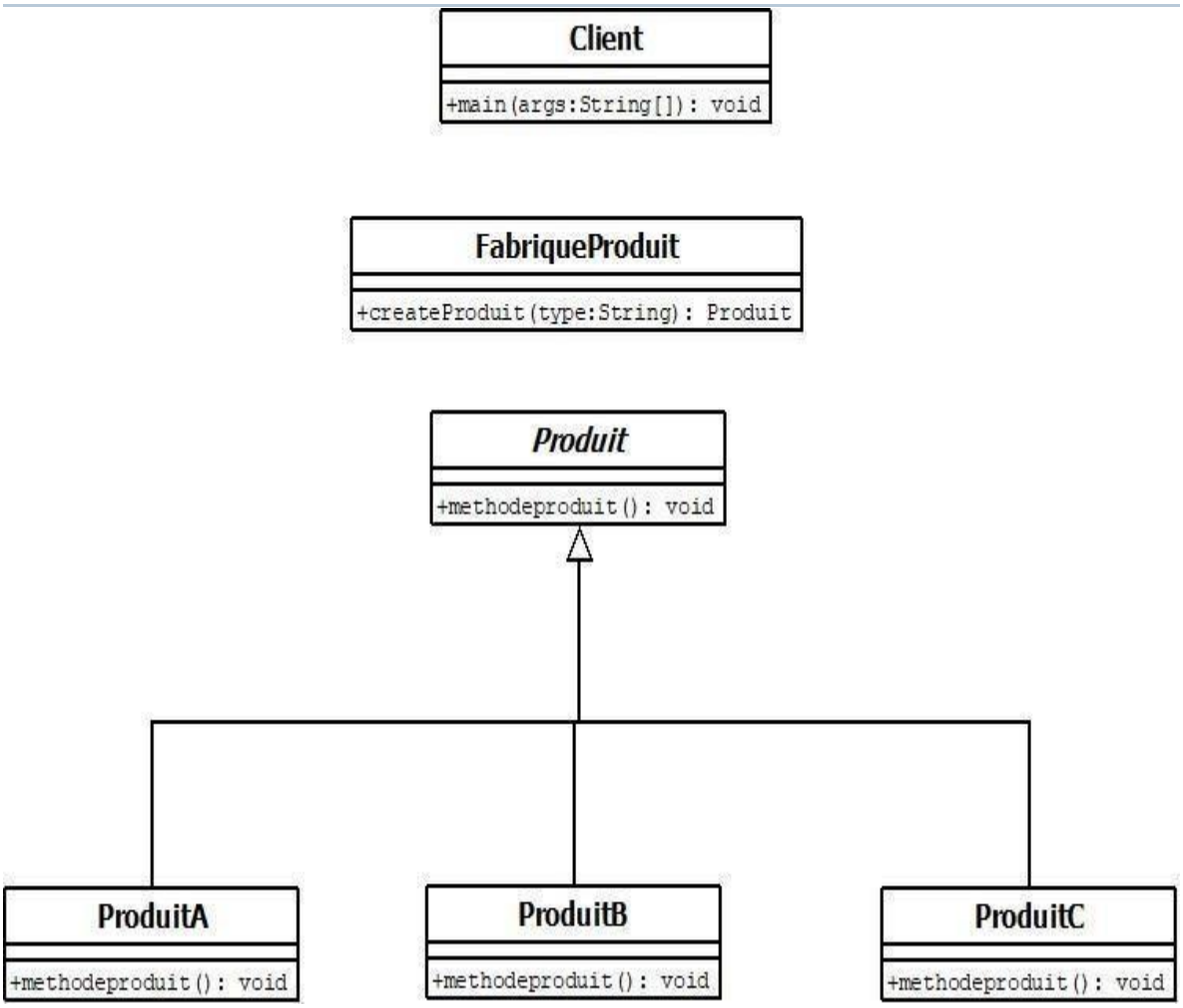


Figure 1 : Diagramme UML de la version du cours.

4. Fonctionnement de la version du cours

La version initiale définit trois constantes : typeA, typeB et typeC. La méthode createProduit() utilise ces constantes dans une instruction switch pour créer respectivement ProduitA, ProduitB ou ProduitC.

Lorsque le client demande un type qui n'existe pas, la fabrique déclenche une exception IllegalArgumentException. Le client utilise un bloc try/catch pour intercepter cette erreur et afficher un message au lieu de laisser le programme s'arrêter brutalement.

Dans cette version, le client teste les produits A, B et C, puis essaie de demander le produit D. Ce dernier n'étant pas encore disponible, une erreur contrôlée est affichée.

5. Modifications apportées

		A, B, C et D
Produits disponibles	A, B et C	
Création des objets	Affectation à une variable puis retour	Retour direct des objets dans le switch
Produit D	Absent	Ajout de ProduitD.java et du typeD

Message d'erreur	Message générique	Le type inconnu est précisé
Annotation	Pas d'annotation @Override dans les classes	@Override ajouté dans ProduitC et ProduitD

La principale extension est l'ajout de ProduitD. La fabrique déclare désormais la constante typeD et crée un objet ProduitD lorsque ce type est demandé.

La méthode createProduit() retourne directement le nouvel objet dans chaque branche du switch. Cette écriture évite d'utiliser une variable intermédiaire.

Élément	Version du cours	Version modifiée

L'annotation @Override indique qu'une méthode redéfinit une méthode héritée. Elle permet notamment au compilateur de détecter certaines erreurs de nom ou de signature.

6. Diagramme UML de la version modifiée

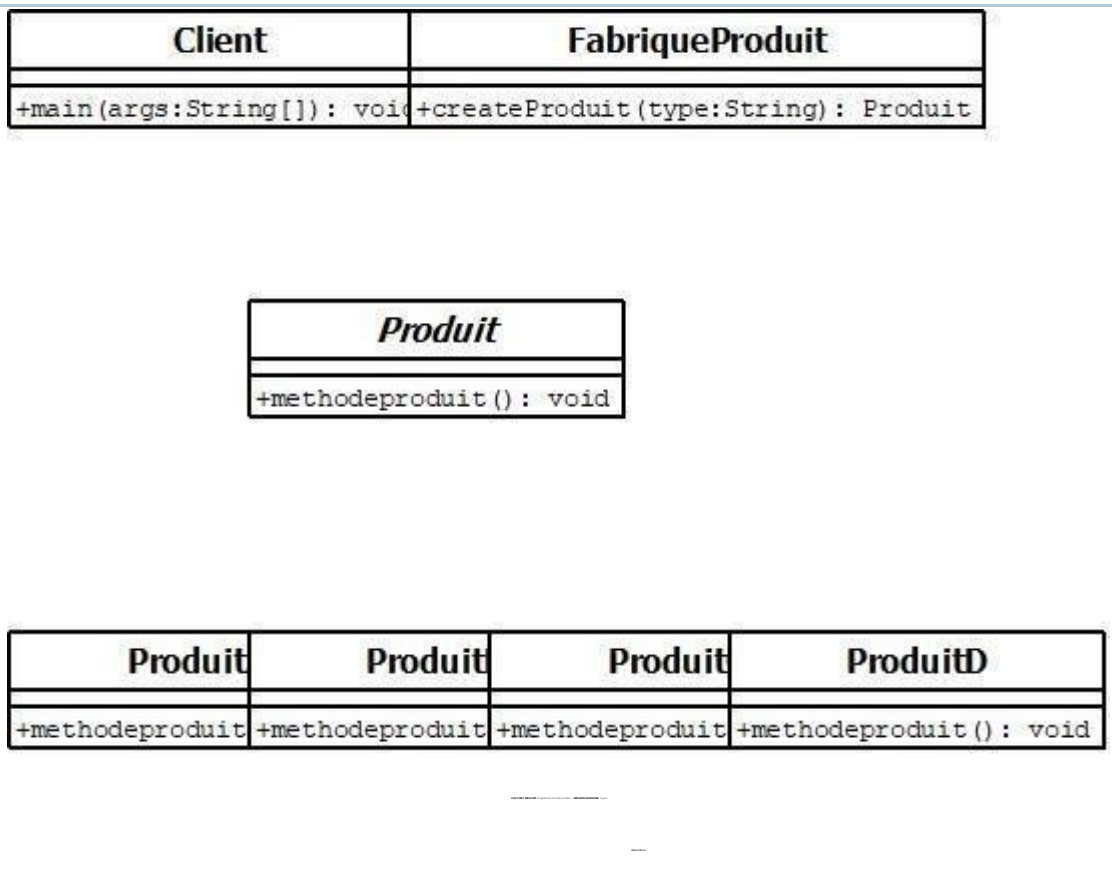


Figure 3 : Diagramme modifié avec les relations et dépendances.

7. Tests et résultats

Les deux versions ont été compilées avec la commande javac *.java, puis exécutées à partir de la classe Client.

Version	Test	Résultat attendu

Cours	Produits A, B et C	Chaque produit affiche son propre message.
Cours	Produit D	Une exception contrôlée signale que le type est inconnu.
Modifiée	Produits A, B, C et D	Chaque produit affiche son propre message.
Modifiée	Type X	Une exception contrôlée précise que le type X est inconnu.

Les compilations et les exécutions des deux versions ont été vérifiées avec succès.

8. Analyse et limites

La fabrique centralise la création des objets. Le client utilise le type abstrait Produit et n'a pas besoin de créer directement les produits concrets. Cette organisation réduit la dépendance du client aux classes concrètes.

L'ajout du produit D montre qu'un nouveau produit peut être intégré sans modifier le code du client. Il faut toutefois ajouter le nouveau type et son cas de création dans FabriqueProduit.

> **Précision technique :** dans la terminologie stricte des patrons de conception GoF, une fabrique unique qui choisit une classe concrète avec une instruction switch est généralement appelée Simple Factory. Le Factory Method classique repose plutôt sur une méthode de création redéfinie par des sous-classes de créateurs. Ce rapport décrit l'implémentation réellement réalisée dans ce projet, conformément à l'intitulé utilisé dans le cours.

9. Conclusion

Ce travail a permis de mettre en place une fabrique de produits en Java, d'utiliser une classe abstraite et plusieurs classes concrètes, puis d'étendre l'application avec un quatrième produit.

La version modifiée prend en charge le produit D, améliore le message d'erreur et utilise l'annotation @Override dans certaines classes. Les deux versions ont été compilées et exécutées avec succès.